N-Body Gravitational Simulator

Rounak Chattopadhyay

May 2026

1 Introduction

An N-body simulator is a simulation that models the dynamical evolution of a system of interacting particles under physical forces, like gravity. In astrophysics, such simulations are used to study systems ranging from planetary motion and star clusters to galaxies and large-scale cosmic structure formation. In this project we discuss one such simple N-body gravitational simulator, implemented in Python.

2 Why Do We Need A Simulator?

We need a simulator to study the motion of particles moving under mutual gravitation. This is because the general gravitational N-body problem does not admit analytical closed-form solutions for $N \geq 3$. Astrophysicists use such simulators for:

- Planetary and Orbital Dynamics
- Star Cluster Evolution
- Galaxy Collisions and Mergers
- Dark Matter and Large Scale Structure Formation
- Galactic Dynamics and Spiral Structure

Various algorithms are employed for such simulators. One of the simplest algorithms, Position Verlet Integration is used in this project.

3 Theory and Numerical Implementation

Particles move according to Newton's Second Law of Motion:

$$m\frac{d^2\boldsymbol{r}}{dt^2} = \boldsymbol{F}(\boldsymbol{r})$$

where, the force $\boldsymbol{F}(\boldsymbol{r})$ is given by Newton's Law of Universal Gravitation:

$$\boldsymbol{F}(\boldsymbol{r}) = -G\frac{Mm}{r^2}\hat{\boldsymbol{r}}$$

Now, let us suppose we have N particles. Then for the i^{th} particle, we have

$$\frac{d^2\boldsymbol{r_i}}{dt^2} = -G\sum_{\substack{j=1 \\ j\neq i}}^{N} \frac{m_j}{\|\boldsymbol{r_i} - \boldsymbol{r_j}\|^3}(\boldsymbol{r_i} - \boldsymbol{r_j})$$

Since there exists no closed form solution of this equation when $N \geq 3$, we solve this numerically. So, it is necessary to change the differential equations into difference equations. We discretise our equation as below:

$$\frac{d^2\boldsymbol{r}(t)}{dt^2} \approx \frac{\boldsymbol{r}^{(n+1)} - 2\boldsymbol{r}^{(n)} + \boldsymbol{r}^{(n-1)}}{(\Delta t)^2}$$

where Δt is the time step for one iteration, and hence $\boldsymbol{r}^{(n)} = \boldsymbol{r}(t = n\Delta t)$, n denotes the iteration step.

Our project forms a 2D simulation. We use direct summation together with Position Verlet integration to preserve numerical stability and energy conservation over long simulations. The force exerted on a particle i is computed as the sum of the forces from all other particles $j \neq i$ inside the system. This technique gives the correct acceleration for every particle, but has the major drawback that $O(N^2)$ operations are required for the calculation. For simulations of collisionless systems, a small error in the accelerations is, however, tolerable without affecting the evolution of the system. There exist several more sophisticated methods, such as Barnes-Hut tree algorithms, that reduce the computational complexity while introducing controlled approximations in the force calculation. In this project, however, we use direct summation together with Position Verlet integration for simplicity and improved long-term numerical stability.

We follow the simple steps:

1. We input N and all the initial parameters like masses of the particles, initial x and y co-ordinates and initial velocity along x and y directions. Thus our particle is represented as a class containing the five parameters (m, x, y, v_x, v_y).

2. We compute the gravitational force on each particle. Force on particle i is given as:

$$\boldsymbol{F}_i = -G \sum_{\substack{j=1 \\ j \neq i}}^{N} \frac{m_i m_j}{(\|\boldsymbol{r}_i - \boldsymbol{r}_j\|^2 + \varepsilon^2)^{\frac{3}{2}}} (\boldsymbol{r}_i - \boldsymbol{r}_j)$$

   Observe here we have introduced a softening constant ε in this formula, so that when two particles come really close the forces do not diverge. In our simulation we have used $\varepsilon = 10^{-12}$.

3. We input the time step size Δt and total time T of the simulation.

4. Finally we write our iterative equation for the trajectory:

$$\boldsymbol{r}_i^{(n+1)} = 2\boldsymbol{r}_i^{(n)} - \boldsymbol{r}_i^{(n-1)} + \boldsymbol{a}_i(\boldsymbol{r}_i^{(n)})(\Delta t)^2$$

   where,

$$\boldsymbol{a}_i(\boldsymbol{r}_i^{(n)}) = \frac{\boldsymbol{F}_i(\boldsymbol{r}_i^{(n)})}{m_i}$$

   with our base cases,

$$\boldsymbol{r}_i^{(1)} = \boldsymbol{r}_i^{(0)} + \boldsymbol{v}_i^{(0)}\Delta t + \frac{1}{2}\boldsymbol{a}_i(\boldsymbol{r}_i^{(0)})(\Delta t)^2$$

   for given initial parameters $\boldsymbol{r}_i^{(0)}$ as the initial position vector, and $\boldsymbol{v}_i^{(0)}$ as the initial velocity vector.

This way we iteratively compute and plot the trajectory of the particles. Once we are done with the individual trajectories, we determine the trajectory of the centre of mass by:

$$\boldsymbol{r}_{COM}^{(n)} = \frac{\sum_{i=1}^{N} m_i \boldsymbol{r}_i^{(n)}}{\sum_{i=1}^{N} m_i}$$

The determination and plotting of the centre of mass is important. If the trajectory of the centre of mass is a straight line, then the momentum of the system is conserved: which is a sign our simulator is doing good.

4 Error Term

To determine the error term order in Position Verlet integration, we Taylor Expand the terms:

$$\boldsymbol{r}_i^{(n+1)} = \boldsymbol{r}_i^{(n)} + \frac{d\boldsymbol{r}_i^{(n)}}{dt}\Delta t + \frac{1}{2!}\frac{d^2\boldsymbol{r}_i^{(n)}}{dt^2}(\Delta t)^2 + \frac{1}{3!}\frac{d^3\boldsymbol{r}_i^{(n)}}{dt^3}(\Delta t)^3 + \frac{1}{4!}\frac{d^4\boldsymbol{r}_i^{(n)}}{dt^4}(\Delta t)^4 + \frac{1}{5!}\frac{d^5\boldsymbol{r}_i^{(n)}}{dt^5}(\Delta t)^5 + \frac{1}{6!}\frac{d^6\boldsymbol{r}_i^{(n)}}{dt^6}(\Delta t)^6 + ...$$

$$\boldsymbol{r}_i^{(n-1)} = \boldsymbol{r}_i^{(n)} - \frac{d\boldsymbol{r}_i^{(n)}}{dt}\Delta t + \frac{1}{2!}\frac{d^2\boldsymbol{r}_i^{(n)}}{dt^2}(\Delta t)^2 - \frac{1}{3!}\frac{d^3\boldsymbol{r}_i^{(n)}}{dt^3}(\Delta t)^3 + \frac{1}{4!}\frac{d^4\boldsymbol{r}_i^{(n)}}{dt^4}(\Delta t)^4 - \frac{1}{5!}\frac{d^5\boldsymbol{r}_i^{(n)}}{dt^5}(\Delta t)^5 + \frac{1}{6!}\frac{d^6\boldsymbol{r}_i^{(n)}}{dt^6}(\Delta t)^6 + ...$$

and we add them :

$$\boldsymbol{r}_i^{(n+1)} + \boldsymbol{r}_i^{(n-1)} = 2\boldsymbol{r}_i^{(n)} + \frac{d^2\boldsymbol{r}_i^{(n)}}{dt^2}(\Delta t)^2 + \frac{1}{12}\frac{d^4\boldsymbol{r}_i^{(n)}}{dt^4}(\Delta t)^4 + O((\Delta t)^6)$$

Comparing with the equation we had, the local truncation error term is $\frac{1}{12}\frac{d^4\boldsymbol{r}_i^{(n)}}{dt^4}(\Delta t)^4$. The global truncation error is $O((\Delta t)^2)$.
The Verlet integrator is often preferred in long-term physical simulations because it preserves the geometric structure of motion much better than methods like Euler and even RK4. Unlike the simple Euler method, which rapidly accumulates energy error and can cause orbits or oscillations to spiral outward or collapse artificially, Verlet is time-reversible and symplectic, meaning it approximately conserves the total energy and phase-space structure of Hamiltonian systems over long durations. Although RK4 is far more accurate per individual timestep, it is not symplectic, so in very long simulations its small energy errors can systematically drift, eventually producing nonphysical behavior. Verlet therefore becomes especially valuable in problems such as gravitational N-body systems, molecular dynamics, and orbital mechanics, where maintaining stable qualitative motion over millions of time-steps matters more than achieving extremely high local accuracy at each step. It also has relatively low computational cost since it requires only one force evaluation per timestep in its basic form, making it efficient for large simulations.

5 Code Availability

The code for this project is available here.

6 References

1. Wikipedia: Verlet Integration
2. K. Dolag and T. Hoffmann, *N-body Simulations and Galaxy Formation*, Computational Methods in Astrophysics, Department of Physics, Ludwig-Maximilians-Universität München, Version 0.42, 30 Oct. 2024.
3. *Astrophysics: N-Body Simulation*, 3rd Year Physics Laboratories, University of Melbourne, compiled Jan. 15, 2018.